

GIẢI THÍCH WEIGHTS & DATAPATH

Model: drowsiness_asic_int8.tflite | INT8 Quantized CNN

Dùng để thiết kế ASIC / RTL — PE Array Datapath

1. TỔNG QUAN KIẾN TRÚC MODEL

Model này là một CNN (Convolutional Neural Network) INT8 đầy đủ gồm 7 layer chính, thiết kế để chạy trên ASIC với phép tính tích chập nguyên (integer-only arithmetic). Toàn bộ weight và bias đã được quantize, không có phép tính floating-point nào trong inference path.

#	Layer	Input Shape	Output Shape	Weights	Tham số
1	Conv1 (3×3, 32 filter)	[1, 128, 128, 1]	[1, 64, 64, 32]	[32, 3, 3, 1] = 288 B	288 + 32×4 B
2	Conv2 (3×3, 64 filter)	[1, 64, 64, 32]	[1, 32, 32, 64]	[64, 3, 3, 32] = 18,432 B	18K + 64×4 B
3	Conv3 (3×3, 128 filter)	[1, 32, 32, 64]	[1, 16, 16, 128]	[128, 3, 3, 64] = 73,728 B	72K + 128×4 B
4	Conv4 (3×3, 128 filter)	[1, 16, 16, 128]	[1, 8, 8, 128]	[128, 3, 3, 128] = 147,456 B	144K + 128×4 B
5	Global Avg Pool (GAP)	[1, 8, 8, 128]	[1, 1, 1, 128]	Không có weight	0
6	FC1 Dense (128→32)	[1, 128]	[1, 32]	[32, 128] = 4,096 B	4K + 32×4 B
7	FC2 Dense (32→1) Output	[1, 32]	[1, 1]	[1, 32] = 32 B	32 + 1×4 B
TOTAL				Weight SRAM: ~254 KB	Bias ROM: 1.5 KB

- ❖ Mỗi Conv layer đều có: MaxPool 2×2 stride=2 phía sau → giảm kích thước feature map xuống 1/2 mỗi lần.
- ❖ Không có Batch Normalization riêng — BN đã được fold vào weight và bias khi quantize.
- ❖ ReLU được thực hiện ở bước clip sau requantization (clip về [0, 127] thay vì [-128, 127]).

2. QUANTIZATION — SCALE & ZERO POINT

2.1 Ý nghĩa của Scale và Zero Point

Trong INT8 quantization, mỗi số thực **x_real** được biểu diễn qua số nguyên **x_int8** theo công thức:

```
x_real = scale × (x_int8 - zero_point)
x_int8 = round(x_real / scale) + zero_point [clip về [-128, 127]]
```

Scale quyết định độ phân giải (1 LSB tương đương bao nhiêu đơn vị thực). **Zero point** dịch chuyển điểm 0 về vị trí phù hợp trong dải [-128, 127].

2.2 Quantization Parameters của model này

Tensor	Scale	Zero Point	Ý nghĩa
Input (serving_default_input_q07:0)	0.003922	-128	1 LSB $\approx$ 0.39% cường độ ảnh
Output (StatefulPartitionedCall:0)	0.066526	-2	1 LSB $\approx$ 0.067 logit unit
Conv1 weights (per-channel)	0.00934 $\div$ 0.04399	0	Mỗi output channel có scale riêng
FC1 activation sau GAP	0.016025	0	Scale của GAP output

⚠ Input zero point = -128 $\rightarrow$ pixel int8 range là [-128, 127] tương đương float [0.0, ~1.0].
Pixel uint8 [0, 255] $\rightarrow$ right-shift 1 $\rightarrow$ int8 [0, 127] $\rightarrow$ đây là Q0.7 format.
Khi đưa vào TFLite: float32 = int8_q07 / 128.0 (SCALE_FACTOR = 128.0).

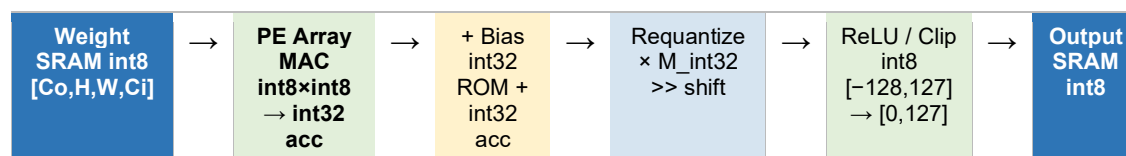
2.3 Per-Channel Quantization của Conv weights

Các layer Conv sử dụng **per-channel quantization** — mỗi output channel (filter) có một scale riêng. Điều này cho phép mỗi filter được biểu diễn với độ chính xác tối ưu.

Layer	Num Channels	Scale Min	Scale Max	Ratio Max/Min
Conv1	32	0.009343	0.043993	4.7×
Conv2	64	0.000270	0.002196	8.1×
Conv3	128	0.000342	0.001381	4.0×
Conv4	128	0.000408	0.001189	2.9×
FC1 (Dense)	32	0.001552	0.003307	2.1×

3. DATAPATH: LOAD WEIGHT $\rightarrow$ PE $\rightarrow$ REQUANT $\rightarrow$ LAYER TIẾP THEO

3.1 Tổng quan luồng dữ liệu qua một Conv Layer



3.2 Bước 1 — Load Weight từ SRAM vào PE

Weight layout trong TFLite INT8 (OHWI order): Tensor shape = [Cout, H, W, Cin]

Để tính output pixel (out_row, out_col), output channel co, PE cần load:

```
// Conv1 làm ví dụ: shape [32, 3, 3, 1]
// Mỗi lần tính 1 output neuron:
```

```

for (co = 0; co < 32; co++) {
    acc = 0;
    for (h = 0; h < 3; h++) {
        for (w = 0; w < 3; w++) {
            for (ci = 0; ci < 1; ci++) {
                weight_val = W[co][h][w][ci]; // load từ Weight SRAM
                input_val = X[in_row+h][in_col+w][ci]; // load từ Input SRAM
                acc += (int32)(weight_val) * (int32)(input_val - inp_zp);
            }
        }
    }
    acc += bias[co]; // load từ Bias ROM
    out[co] = requantize(acc, M_int32[co], shift[co], out_zp);
}

```

- ❖ $\text{inp_zp} = -128$ (zero point của input). Mỗi giá trị input phải trừ đi zero point trước khi nhân.
- ❖ Weight zero point = 0 (per-channel INT8 weight luôn có zp = 0, không cần trừ).
- ❖ Kết quả acc là int32 — cần int32 accumulator trong hardware để không overflow.

3.3 Bước 2 — MAC Operation trong PE

Mỗi PE (Processing Element) thực hiện phép nhân-cộng dồn (Multiply-Accumulate):

```

// 1 MAC step:
x_adj = (int32)input_val - (int32)inp_zp; // inp_zp = -128
product = (int32)weight_val * x_adj; // int8 * int8 → int32
acc += product; // int32 + int32 → int32

// Sau tất cả K×K×Cin MAC:
acc += (int32)bias[co]; // bias là int32

```

Ví dụ cụ thể Conv1, ch=0 (từ REQUANT_PARAMS.txt):

Tham số	Giá trị	Đơn vị	Ghi chú
S_input	0.003922	float32	Scale của input tensor
S_weight[0]	0.009343 (ch 0)	float32	Scale weight, ch=0 của Conv1
inp_zp	-128	int32	Zero point input, trừ trước khi nhân
Bias[0]	(xem *.bias.txt)	int32	Bias đã fold BN, int32

3.4 Bước 3 — Requantization (integer-only, không cần FPU)

Sau khi MAC xong, **acc (int32)** cần được requantize về int8 cho layer tiếp theo. Toàn bộ bước này chỉ dùng phép nhân và shift nguyên:

```

// Requantization cho output channel 'co':
M = S_input * S_weight[co] / S_output_layer

// M được xấp xỉ bằng:  $M \approx M\_int32 / 2^{\text{total\_shift}}$ 
// trong đó  $M\_int32 \approx M\_norm \times 2^{30}$  (Q1.30 fixed-point)

int64 scaled = (int64)acc * (int64)M_int32[co];
int32 shifted = (int32)(scaled >> total_shift);
int8 out = clip(shifted + out_zp, -128, 127);

```

```
// Nếu có ReLU sau layer: clip về [0, 127] thay vì [-128, 127]
int8 relu = (out < 0) ? 0 : out;
```

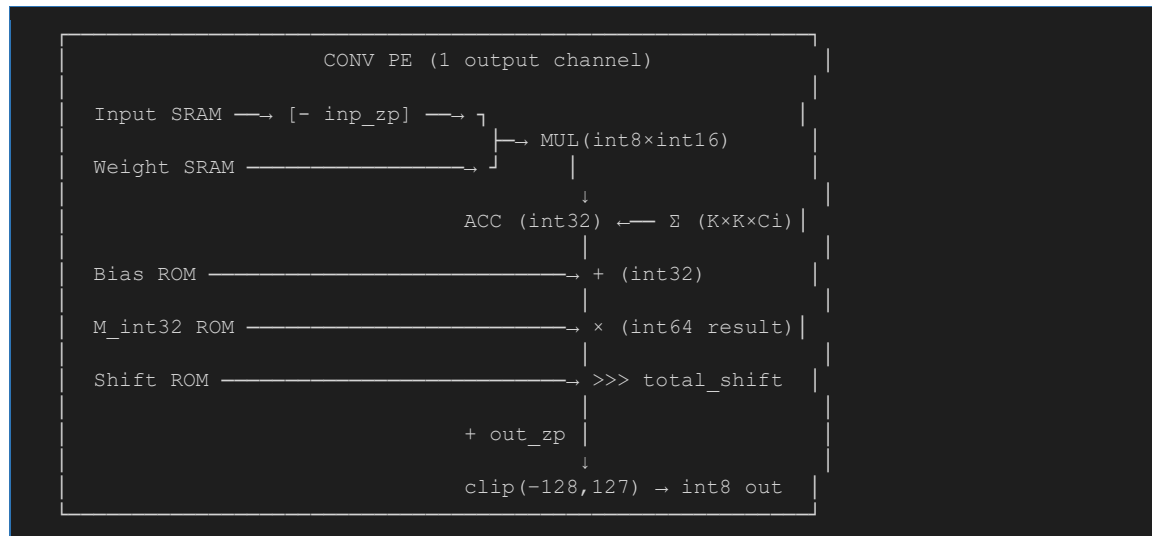
3.5 Bảng Requant Params — Conv1 (32 channels)

Mỗi output channel của Conv1 có bộ params riêng. RTL phải lưu tất cả M_int32 và shift vào ROM:

Ch	M_float	M_int32	Shift	Verilog shift
0	0.001392550881	765562943	39	>>> 39
1	0.000550760294	605567348	40	>>> 40
2	0.001210152695	665288480	39	>>> 39
3	0.001871774462	1029018893	39	>>> 39
4	0.001348619269	741411284	39	>>> 39
5	0.001563640500	859620456	39	>>> 39
6	0.001952106106	1073181681	39	>>> 39
7	0.001168043094	642138482	39	>>> 39
11	0.002593320889	712846618	38	>>> 38
15	0.002244074744	616846569	38	>>> 38
...	...	...	...	...
31	0.001487581698	817806687	39	>>> 39

4. VERILOG RTL — DATAPATH HOÀN CHỈNH

4.1 Sơ đồ khối một Conv PE



4.2 Verilog pseudocode — Requantization

```

// Parameters cho mỗi output channel (lưu trong ROM)
// Conv1, ch=0:
parameter M_INT32      = 32'd765562943;
parameter TOTAL_SHIFT = 6'd39;
parameter OUT_ZP       = -8'sd2;      // model output zp

// Signals
wire signed [31:0] acc;                // kết quả sau MAC + bias
wire signed [63:0] scaled;             // acc × M_int32
wire signed [31:0] shifted;            // sau arithmetic right shift
wire signed [7:0] clipped;             // sau clip [-128, 127]
wire signed [7:0] relu_out;           // sau ReLU (nếu cần)

// Datapath
assign scaled = $signed(acc) * $signed(M_INT32);
assign shifted = $signed(scaled) >>> TOTAL_SHIFT;

// Clip với out_zp
assign clipped = (shifted + OUT_ZP > 127) ? 8'sd127 :
                (shifted + OUT_ZP < -128) ? -8'sd128 :
                (shifted + OUT_ZP);

// ReLU (chỉ áp dụng sau Conv, không phải output cuối)
assign relu_out = clipped[7] ? 8'd0 : clipped; // MSB=1 → âm → 0

```

4.3 Lưu ý quan trọng về bit-width

Tín hiệu	Bit-width	Signed?	Lý do
input_val (sau - inp_zp)	9-bit (int16 safe)	Signed	inp_zp = -128, max = 127-(-128) = 255 → cần 9 bit
weight_val	8-bit (int8)	Signed	range [-128, 127]
acc (sau mỗi MAC step)	32-bit (int32)	Signed	Tránh overflow: max $9 \times 255 \times 127 \approx 290K/\text{step}$ $\times 9 = 2.6M$, cần 22 bit; int32 an toàn
acc sau tất cả MAC	32-bit (int32)	Signed	Conv1: $3 \times 3 \times 1 = 9$ MAC; Conv4: $3 \times 3 \times 128 = 1152$ MAC → max ~147M, int32 OK
scaled = acc × M_int32	64-bit (int64)	Signed	$\text{int32} \times \text{int32} \rightarrow$ cần int64 để không mất bit
shifted (sau >>>)	32-bit	Signed	Lấy 32 bit thấp sau khi shift
output (sau clip)	8-bit (int8)	Signed	range [-128, 127]; sau ReLU: [0, 127]

5. CHI TIẾT TỪNG LAYER

5.1 Conv1 [32 filters, 3×3, stride=1, padding=SAME] → MaxPool 2×2

Weight shape	[32, 3, 3, 1] = $32 \times 9 \times 1 = 288$ bytes int8
Bias shape	[32] = $32 \times 4 = 128$ bytes int32
Input	[1, 128, 128, 1] → 128×128 ảnh grayscale, 1 channel

Output (sau MaxPool)	[1, 64, 64, 32] → 64×64 feature maps, 32 channels
MACs per pixel	3 × 3 × 1 = 9 MACs
Total MACs	128×128×32×9 = 4,718,592 ops
Weight scale range	0.009343 ÷ 0.043993 (4.7× spread → per-channel cần thiết)

5.2 Conv2 [64 filters, 3×3, stride=1] → MaxPool 2×2

Weight shape	[64, 3, 3, 32] = 64 × 9 × 32 = 18,432 bytes int8
Bias shape	[64] = 64 × 4 = 256 bytes int32
Input	[1, 64, 64, 32]
Output (sau MaxPool)	[1, 32, 32, 64]
MACs per pixel per ch	3 × 3 × 32 = 288 MACs
Total MACs	64×64×64×288 = 75,497,472 ops (layer tốn nhất trước Conv3/4)

5.3 Conv3 & Conv4 [128 filters, 3×3]

Conv3 và Conv4 giống nhau về cấu trúc PE, chỉ khác input channel count:

	Conv3	Conv4	Đơn vị
Weight shape	[128, 3, 3, 64]	[128, 3, 3, 128]	int8
Weight size	73,728 B = 72 KB	147,456 B = 144 KB	byte
MACs/pixel/ch	9 × 64 = 576	9 × 128 = 1,152	ops
Total MACs	~603M	~1.2B	ops

5.4 Global Average Pooling (GAP)

GAP lấy trung bình tất cả spatial positions cho mỗi channel. Không có weight convolution — chỉ là phép cộng rồi chia:

```
// Input: [1, 8, 8, 128] → 8×8 spatial, 128 channels
// Output: [1, 1, 1, 128] → 1 giá trị mỗi channel

for (ch = 0; ch < 128; ch++) {
    int32 sum = 0;
    for (h = 0; h < 8; h++)
        for (w = 0; w < 8; w++)
            sum += (int32)input[h][w][ch] - gap_zp; // 64 phép cộng

    // Chia cho 64 = shift phải 6 bit (hoặc dùng requant M_int32)
    output[ch] = requantize(sum, M_gap, shift_gap, gap_out_zp);
}

// GAP scale: 0.016025 (index 17 trong INDEX.json)
```

5.5 FC1 — Fully Connected (128→32) + ReLU

Weight shape	[32, 128] = 4,096 bytes int8 (matrix multiply)
Bias shape	[32] = 128 bytes int32
Input	[1, 128] → 128 giá trị sau GAP

Output	[1, 32] → 32 features (sau ReLU)
MACs	$128 \times 32 = 4,096$ MACs
Activation	ReLU → clip output về [0, 127]

5.6 FC2 — Output Layer (32→1)

Weight shape	[1, 32] = 32 bytes int8
Bias shape	[1] = 4 bytes int32
Output	[1, 1] → 1 logit value (int8)
Output scale	0.066526
Output ZP	-2
Decode logit	$\text{logit} = (\text{out_int8} - (-2)) \times 0.066526 = (\text{out_int8} + 2) \times 0.066526$
Activation	KHÔNG có ReLU — đây là logit thô, sigmoid apply ở CPU/SW sau

6. DECODE OUTPUT — TỪ INT8 → KẾT QUẢ CUỐI

Output của model là 1 giá trị int8. Đưa ra kết quả ACTIVE/FATIGUE:

```
// Bước 1: Dequantize int8 → logit float
out_int8 = model_output;           // ví dụ: -120
out_zp    = -2;
out_scale = 0.066526;
logit     = (out_int8 - out_zp) * out_scale
           = (-120 - (-2)) * 0.066526
           = -118 × 0.066526
           = -7.850 (logit rất âm → prob rất thấp)

// Bước 2: Sigmoid
prob = 1 / (1 + exp(-logit)) = 1 / (1 + exp(7.85)) ≈ 0.0004

// Bước 3: Phân loại
// prob < 0.01 → mắt NHẮM → FATIGUE 😓
// prob >= 0.01 → mắt MỞ → ACTIVE 😊

// (Calibrated từ data thực: mắt mở prob≈0.05-0.95, mắt nhắm prob≈0.0002-0.003)
```

🔴 Trong ASIC: sigmoid không cần tính ở hardware. Chỉ cần so sánh out_int8 với ngưỡng tương đương.
 $\text{prob} < 0.01 \leftrightarrow \text{logit} < \ln(0.01/0.99) \approx -4.60 \leftrightarrow \text{out_int8} < -4.60/0.066526 + (-2) \approx -71$
 → Nếu out_int8 < -71 thì FATIGUE, ngược lại ACTIVE. Không cần FPU!

7. MEMORY MAP CHO ASIC

Memory Block	Size	Dtype	Access	File nguồn
Input SRAM	16,384 B	int8	Read/Write	pixels_hex.txt
Conv1 Weight ROM	288 B	int8	Read-only	conv1.hex.txt

Memory Block	Size	Dtype	Access	File nguồn
Conv1 Bias ROM	128 B	int32	Read-only	conv1.bias.txt
Conv2 Weight ROM	18,432 B	int8	Read-only	conv2.hex.txt
Conv3 Weight ROM	73,728 B	int8	Read-only	conv3.hex.txt
Conv4 Weight ROM	147,456 B	int8	Read-only	conv4.hex.txt
FC1 Weight ROM	4,096 B	int8	Read-only	fc1.hex.txt
FC2 Weight ROM	32 B	int8	Read-only	fc2.hex.txt
M_int32 + Shift ROM	~2 KB	int32	Read-only	REQUANT_PARAMS.txt
Feature Map Buffer	~64 KB	int8	Read/Write	(ping-pong SRAM)
TOTAL ROM (weights)	~244 KB	int8/int32		

- ✦ Feature Map Buffer nên dùng ping-pong SRAM: một bank chứa input layer N, bank kia chứa output layer N → layer N+1 đọc.
- ✦ Weight SRAM có thể nạp từ Flash/DRAM khi power-up hoặc bake thẳng vào ROM cell tùy thiết kế.
- ✦ M_int32 và shift có thể lưu trong cùng ROM với bias nếu địa chỉ được tổ chức hợp lý.

TÓM TẮT DATAPATH

1	Load int8 pixel từ Input SRAM, trừ inp_zp (-128) → signed input
2	Load int8 weight từ Weight ROM, nhân với signed input → int32 partial sum
3	Cộng dồn tất cả K×K×Cin MAC steps → int32 accumulator
4	Cộng int32 bias từ Bias ROM → int32 acc_biased
5	Nhân int64: acc × M_int32[co] → arithmetic right shift total_shift → int32
6	Cộng out_zp rồi clip [-128,127] → int8 output (ReLU: clip [0,127])
7	Ghi int8 vào Output Feature Map SRAM → làm input của layer tiếp theo